

DETECTIVE OS

Interactive Crime Investigation & Evidence Analysis System

CSE2006 — Programming in Java
Project Report Draft • Version 2.0.0

Prepared for VITyarthi Build Your Own Project Assessment

1. Introduction

Detective OS is a desktop Java application that simulates a fictional criminal investigation. It provides a structured workflow for opening a case, examining suspects and evidence, recording statements, reconstructing a timeline, running an explainable reasoning engine and producing an investigation report.

2. Problem Statement

Basic CRUD applications can store case records but do not demonstrate how multiple pieces of evidence and statements can be connected. Detective OS models the investigation workflow as a software system and makes its reasoning rules visible to the user.

3. Objectives

Develop a modular Java application; demonstrate object-oriented programming; implement exception handling and concurrency; use Java Collections and I/O; persist domain objects using JPA/Hibernate; perform direct analytics using JDBC; provide testing and documentation.

4. Functional Requirements

Authentication; case management; procedural case generation; suspect/witness management; evidence locker; interviews and statements; timeline reconstruction; explainable suspicion scoring; concurrent analysis; JDBC analytics; report generation; accusation and case closure.

5. Non-functional Requirements

Usability through a tab-based JavaFX interface; reliability through validation and transaction rollback; maintainability through layered packages; performance through concurrent analysis; basic password protection through hashing; resource management through short-lived EntityManagers and shutdown of executor services.

6. System Architecture

The application follows a layered architecture: JavaFX UI → service layer → reasoning/engine layer → repository/data-access layer → JPA/Hibernate and JDBC → MySQL. Domain models are represented as JPA entities.

7. Design Decisions

JavaFX was selected for a demonstrable desktop interface. JPA/Hibernate handles ordinary entity persistence and ORM. JDBC is used for direct reporting queries to demonstrate the course requirement. An EvidenceAnalyzer interface allows different analysis engines to share a common contract. Person is an abstract base class for Suspect, Witness and Victim.

8. Reasoning Engine

Suspicion is a transparent weighted score: documented motive +20, linked evidence up to +30, timeline involvement up to +20 and statement contradictions up to +30. The maximum is 100. The score is a fictional educational indicator and is not a real forensic conclusion.

9. Multithreading

Evidence, timeline and statement analysis are dispatched through a fixed ExecutorService thread pool. Statement analysis synchronizes access to the shared case state, while the other tasks work on read-only or copied data. Final

scoring occurs after the worker tasks complete.

10. Database Design

Core entities include User, CaseFile, Person hierarchy, Evidence, Statement, TimelineEvent, Investigation and Accusation. Hibernate creates/updates the schema for the assessment build. The supplied SQL creates the database and application user.

11. Testing Approach

JUnit tests cover validation and suspicion scoring. The manual test plan covers login, duplicate evidence, reliability validation, timeline ordering, contradiction detection, concurrent analysis, accusation, closed-case protection, report generation and JDBC analytics.

12. Screenshots / Results

INSERT FINAL SCREENSHOTS HERE: login screen, case dashboard, suspect intelligence, evidence locker, timeline, reasoning output, report generation and JDBC analytics.

13. Challenges Faced

Coordinating shared case state during concurrent analysis, keeping the reasoning rules explainable, mapping object relationships to a relational database and keeping the JavaFX interface responsive were key implementation challenges.

14. Learnings & Key Takeaways

The project demonstrates how Java OOP, Collections, exceptions, threads, synchronization, I/O, JDBC and JPA can be combined into one coherent application rather than implemented as isolated examples.

15. Future Enhancements

Add richer procedural case generation, graph visualization for evidence relationships, role-based permissions, richer analytics dashboards, database migrations and a more advanced rule engine.

16. References

Herbert Schildt, Java: The Complete Reference, 11th edition. Oracle Java documentation. Jakarta Persistence API documentation. Hibernate ORM documentation. JUnit 5 documentation.

Appendix A — CSE2006 Mapping

Course topic	Project implementation
OOP	CaseFile, Person hierarchy, services and engines
Inheritance / polymorphism	Person → Suspect/Witness/Victim
Interfaces	EvidenceAnalyzer
Exceptions	Custom domain exceptions + validation
Multithreading	Three concurrent analysis tasks + synchronization
Collections	ArrayList/List/Stack and collection processing
I/O	Case report and investigation log generation
JDBC	JDBCAalyticsService

JPA / ORM	JPA entities + Hibernate repositories
Testing	JUnit 5